B2B E-commerce User & Vendor Management API

Overview

This document provides comprehensive API documentation for the User and Vendor Management System. The API supports user authentication, profile management, vendor operations, and administrative functions with role-based access control.

Base URL: `{{http://localhost:3000/api/v1/}}` (configured in Postman environment)

---

1. Postman Environment Setup

1.1 Environment Variables

Create a Postman environment with the following variables:

Variable	Value	Description
`baseUrl`	`http://localhost:3000`	API base URL
`accessToken`	*auto-populated*	JWT access token for buyers
`vendorAccessToken`	*auto-populated*	JWT access token for vendors
`adminAccessToken`	*auto-populated*	JWT access token for admins
`inviteToken`	*auto-populated*	Vendor invitation token
`userId`	*auto-populated*	Created user ID
`vendorId`	*auto-populated*	Created vendor ID
`vendorSlug`	*auto-populated*	Vendor slug from public list

1.2 Global Headers

For JSON requests, set these headers:

- `Content-Type: application/json`
- `Authorization: Bearer {{accessToken}}` (for buyer endpoints)
- `Authorization: Bearer {{vendorAccessToken}}` (for vendor endpoints)
- `Authorization: Bearer {{adminAccessToken}}` (for admin endpoints)

2. API Endpoints

2.1 Authentication Endpoints

Method	Endpoint	Authentication	Request Body	Description

POST	`/api/v1/auth/register`	None	User registration data	Register buyer and send OTP
POST	`/api/v1/auth/verify-phone`	Optional	Phone verification data	Verify OTP for registration/phone change
POST	`/api/v1/auth/login`	None	Login credentials	Authenticate user and get tokens
POST	`/api/v1/auth/refresh`	None (cookie/body)	Refresh token	Rotate access token
POST	`/api/v1/auth/logout`	None (cookie)	Empty	Revoke refresh token
POST	`/api/v1/auth/vendor/register`	None	Vendor registration + invite token	Register vendor with admin invite

2.2 User Management Endpoints

Method	Endpoint	Authentication	Request Body	Description
GET	`/api/v1/user/me`	Bearer token	None	Get current user profile
PATCH	`/api/v1/user/me`	Bearer token	Profile update fields	Update user profile
POST	`/api/v1/user/change-password`	Bearer token	Password data	Change user password
POST	`/api/v1/user/phone/change`	Bearer token	New phone number	Request phone change OTP
POST	`/api/v1/user/profile-image`	Bearer token	Multipart file	Upload profile image

2.3 Vendor Endpoints

Public Vendor Endpoints

Method	Endpoint	Authentication	Parameters	Description
GET	`/api/v1/vendors`	None	None	List all active vendors
GET	`/api/v1/vendors/:slug`	None	Vendor slug	Get vendor details by slug

Protected Vendor Endpoints

Method	Endpoint	Authentication	Request Body	Description

GET	`/api/v1/vendor/me`	Vendor token	None	Get vendor profile
PATCH	`/api/v1/vendor/me`	Vendor token	Vendor update data	Update vendor profile
POST	`/api/v1/vendor/logo`	Vendor token	Multipart file	Upload vendor logo
POST	`/api/v1/vendor/banner`	Vendor token	Multipart file	Upload vendor banner

2.4 Administrative Endpoints

Method	Endpoint	Authentication	Request Body	Description
GET	`/api/v1/admin/vendors`	Admin token	None	List all vendors
GET	`/api/v1/admin/vendors/:id`	Admin token	None	Get vendor by ID
POST	`/api/v1/admin/vendors/:id/verify`	Admin token	None	Verify vendor
PATCH	`/api/v1/admin/vendors/:id/status`	Admin token	Status update	Update vendor status
POST	`/api/v1/admin/vendors/invite`	Admin token	Invite data	Generate vendor invite
GET	`/api/v1/admin/vendors/pending`	Admin token	None	List pending vendors
PATCH	`/api/v1/admin/vendors/:id/approve`	Admin token	None	Approve vendor
PATCH	`/api/v1/admin/users/:id/role`	Super Admin token	Role data	Update user role

---

3. API Request Details

3.1 User Registration

Endpoint: `POST /api/v1/auth/register`

Headers: `Content-Type: application/json`

Request Body:

```
{
  "email": "user@example.com",
  "password": "SecurePass123",
  "phone": "+1234567890",
  "firstName": "John",
  "lastName": "Doe",
  "street": "123 Main St",
  "city": "New York",
  "state": "NY",
  "postalCode": "10001",
```

```
  "country": "USA"
}
```

Success Response:

```
{
  "success": true,
  "message": "Verification code sent to your phone.",
  "verificationCode": "123456"
}
```

Error Responses:

- `400` - Validation failed
- `400` - User with this email or phone already exists
- `400` - Verification code already sent

3.2 Phone Verification

Endpoint: `POST /api/v1/auth/verify-phone`

For Registration:

```
{
  "phone": "+1234567890",
  "code": "123456"
}
```

For Phone Change (requires authentication):

```
{
  "code": "123456"
}
```

Success Response:

```
{
  "success": true,
  "message": "Phone verified. Account created successfully."
}
```

3.3 User Login

Endpoint: `POST /api/v1/auth/login`

Request Body:

```
{
  "email": "user@example.com",
  "password": "SecurePass123"
}
```

Success Response:

```
{
  "success": true,
  "data": {
```

```
    "accessToken": "eyJhbGci...",
    "user": {
      "id": "user-id",
      "email": "user@example.com",
      "role": "BUYER"
    }
  }
}
```

Error Responses:

- `401` - Invalid credentials
- `403` - Phone not verified
- `403` - Vendor account pending approval

3.4 Token Refresh

Endpoint: `POST /api/v1/auth/refresh`

Request Body (optional):

```
{
  "refreshToken": "refresh-token-string"
}
```

Success Response:

```
{
  "success": true,
  "data": {
    "accessToken": "new-access-token"
  }
}
```

Error Responses:

- `400` - Refresh token required
- `401` - Invalid refresh token

3.5 User Logout

Endpoint: `POST /api/v1/auth/logout`

Request Body: None

Success Response:

```
{
  "success": true,
  "message": "Logged out successfully"
}
```

3.6 Vendor Registration

Endpoint: `POST /api/v1/auth/vendor/register`

Request Body:

```
{
  "password": "SecurePass123",
  "phone": "+1234567890",
  "companyName": "ACME Corporation",
  "taxId": "123456789",
  "city": "New York",
  "state": "NY",
  "country": "USA",
  "street": "456 Business Ave",
  "postalCode": "10002",
  "website": "https://acme.com",
  "description": "Leading B2B solutions provider",
  "inviteToken": "vendor-invite-token"
}
```

Success Response:

```
{
  "success": true,
  "message": "Verification code sent to your phone.",
  "verificationCode": "654321"
}
```

Error Responses:

- `400` - Invalid or expired invite token
- `400` - User with this email or phone already exists
- `400` - Validation failed

3.7 User Profile Management

Get User Profile: `GET /api/v1/user/me`

- Headers: `Authorization: Bearer {{accessToken}}`

Update User Profile: `PATCH /api/v1/user/me`

```
{
  "firstName": "John",
  "lastName": "Smith",
  "companyName": "ACME Corp"
}
```

Change Password: `POST /api/v1/user/change-password`

```
{
  "oldPassword": "OldPass123",
  "newPassword": "NewPass456"
}
```

Upload Profile Image: `POST /api/v1/user/profile-image`

- Headers: `Authorization: Bearer {{accessToken}}`

- Content-Type: `multipart/form-data`
- Form field: `image` (file)

3.8 Vendor Operations

Public Vendor List: `GET /api/v1/vendors`

- No authentication required

Get Vendor by Slug: `GET /api/v1/vendors/:slug`

- Path parameter: `vendor-slug`

Vendor Profile: `GET /api/v1/vendor/me`

- Headers: `Authorization: Bearer {{vendorAccessToken}}`

Update Vendor Profile: `PATCH /api/v1/vendor/me`

```
{
  "legalName": "ACME Corporation LLC",
  "description": "Updated company description",
  "yearFounded": 2020,
  "employeeCount": "50-100"
}
```

Upload Vendor Logo: `POST /api/v1/vendor/logo`

- Headers: `Authorization: Bearer {{vendorAccessToken}}`
- Content-Type: `multipart/form-data`
- Form field: `logo` (file)

4.9 Administrative Operations

Generate Vendor Invite: `POST /api/v1/admin/vendors/invite`

```
{
  "email": "vendor@company.com",
  "vendorName": "New Vendor Company"
}
```

List All Vendors: `GET /api/v1/admin/vendors`

- Headers: `Authorization: Bearer {{adminAccessToken}}`

Approve Vendor: `PATCH /api/v1/admin/vendors/:id/approve`

- Headers: `Authorization: Bearer {{adminAccessToken}}`

Update Vendor Status: `PATCH /api/v1/admin/vendors/:id/status`

```
{
  "status": "ACTIVE"
}
```

4. Step-by-Step Postman Testing Guide

4.1 Complete Buyer Registration & Testing Flow

Step 1: Register New Buyer

Request:

- Method: `POST`
- URL: `{{baseUrl}}/api/v1/auth/register`
- Headers: `Content-Type: application/json`
- Body:

```
{
  "email": "testbuyer@example.com",
  "password": "BuyerPass123",
  "phone": "+1234567890",
  "firstName": "Test",
  "lastName": "Buyer"
}
```

Expected Response:

```
{
  "success": true,
  "message": "Verification code sent to your phone.",
  "verificationCode": "123456"
}
```

Step 2: Verify Phone Number

Request:

- Method: `POST`
- URL: `{{baseUrl}}/api/v1/auth/verify-phone`
- Headers: `Content-Type: application/json`
- Body:

```
{
  "phone": "+1234567890",
  "code": "123456"
}
```

Expected Response:

```
{
  "success": true,
  "message": "Phone verified. Account created successfully."
}
```

Step 3: Login as Buyer

Request:

- Method: `POST`
- URL: `{{baseUrl}}/api/v1/auth/login`
- Headers: `Content-Type: application/json`
- Body:

```
{
  "email": "testbuyer@example.com",
  "password": "BuyerPass123"
}
```

Expected Response:

```
{
  "success": true,
  "data": {
    "accessToken": "eyJhbGci...",
    "user": {
      "id": "user-uuid",
      "email": "testbuyer@example.com",
      "role": "BUYER"
    }
  }
}
```

Step 4: Get User Profile

Request:

- Method: `GET`
- URL: `{{baseUrl}}/api/v1/user/me`
- Headers: `Authorization: Bearer {{accessToken}}`

Expected Response:

```
{
  "success": true,
  "data": {
    "id": "user-uuid",
    "email": "testbuyer@example.com",
    "firstName": "Test",
    "lastName": "Buyer",
    "role": "BUYER",
    "status": "ACTIVE"
  }
}
```

4.2 Complete Vendor Registration & Testing Flow

Step 1: Admin Generates Vendor Invite

Request:

- Method: `POST`
- URL: `{{baseUrl}}/api/v1/admin/vendors/invite`
- Headers: `Authorization: Bearer {{adminAccessToken}}`
- Body:

```
{
  "email": "testvendor@company.com",
  "vendorName": "Test Vendor Company"
}
```

Expected Response:

```
{
  "success": true,
  "message": "Vendor invite generated successfully.",
  "data": {
    "token": "invite-token-string",
    "inviteLink": "https://.../vendor/register?invite=invite-token-string",
    "expiresIn": "14 days"
  }
}
```

Step 2: Vendor Registers with Invite

Request:

- Method: `POST`
- URL: `{{baseUrl}}/api/v1/auth/vendor/register`
- Headers: `Content-Type: application/json`
- Body:

```
{
  "password": "VendorPass123",
  "phone": "+1234567891",
  "companyName": "Test Vendor Company",
  "taxId": "987654321",
  "city": "New York",
  "state": "NY",
  "country": "USA",
  "inviteToken": "{{inviteToken}}"
}
```

Step 3: Verify Vendor Phone

Request:

- Method: `POST`
- URL: `{{baseUrl}}/api/v1/auth/verify-phone`
- Headers: `Content-Type: application/json`
- Body:

```
{
  "phone": "+1234567891",
  "code": "123456"
}
```

Step 4: Admin Approves Vendor

Request:

- Method: `PATCH`
- URL: `{{baseUrl}}/api/v1/admin/vendors/{{vendorId}}/approve`
- Headers: `Authorization: Bearer {{adminAccessToken}}`

Step 5: Vendor Login

Request:

- Method: `POST`
- URL: `{{baseUrl}}/api/v1/auth/login`
- Headers: `Content-Type: application/json`
- Body:

```
{
  "email": "testvendor@company.com",
  "password": "VendorPass123"
}
```

---

5. Response Schemas

5.1 Standard API Response

Success Response:

```
{
  "success": true,
  "data": { ... }
}
```

Error Response:

```
{
  "success": false,
  "message": "Description of error",
```

```
  "errors": [
    { "path": "field", "message": "detail" }
  ]
}
```

5.2 Authentication Response Schema

```
{
  "success": true,
  "data": {
    "accessToken": "jwt-token-string",
    "user": {
      "id": "user-uuid",
      "email": "user@example.com",
      "role": "BUYER|VENDOR|ADMIN|SUPER_ADMIN"
    }
  }
}
```

5.3 User Profile Schema

```
{
  "id": "user-uuid",
  "email": "user@example.com",
  "firstName": "John",
  "lastName": "Doe",
  "phone": "+1234567890",
  "companyName": "ACME Corp",
  "street": "123 Main St",
  "city": "New York",
  "state": "NY",
  "postalCode": "10001",
  "country": "USA",
  "role": "BUYER",
  "status": "ACTIVE",
  "emailVerified": true,
  "createdAt": "2026-04-16T00:00:00.000Z"
}
```

5.4 Vendor Public Schema

```
{
  "id": "vendor-uuid",
  "name": "ACME Corporation",
  "slug": "acme-corporation",
  "description": "Leading B2B solutions provider",
  "logoUrl": "https://example.com/logo.jpg",
  "bannerUrl": "https://example.com/banner.jpg",
  "yearFounded": 2020,
  "employeeCount": "50-100",
  "productCount": 25,
  "isVerified": true
}
```

5.5 Vendor Private Schema

Includes all public fields plus:

```
{
  "legalName": "ACME Corporation LLC",
  "email": "vendor@acme.com",
  "phone": "+1234567890",
  "website": "https://acme.com",
  "status": "ACTIVE",
  "verifiedAt": "2026-04-16T00:00:00.000Z",
  "minOrderValue": 100,
  "createdAt": "2026-04-16T00:00:00.000Z"
}
```

---

6. Error Handling Guide

6.1 Common HTTP Status Codes

Status Code	Description	Common Causes
`200`	Success	Request completed successfully
`400`	Bad Request	Validation errors, missing required fields
`401`	Unauthorized	Missing/invalid token, inactive user
`403`	Forbidden	Insufficient permissions, phone not verified
`404`	Not Found	Resource doesn't exist
`429`	Too Many Requests	Rate limit exceeded
`500`	Internal Server Error	Server-side issues

6.2 Validation Error Format

```
{
  "success": false,
  "message": "Validation failed",
  "errors": [
    { "path": "email", "message": "Invalid email format" },
    { "path": "password", "message": "Password must be at least 8 characters" }
  ]
}
```

6.3 Common Error Scenarios

Authentication Errors:

- `401` - Authorization token missing
- `401` - Invalid or expired token
- `403` - Phone number not verified
- `403` - Vendor account pending approval

Validation Errors:

- `400` - Email already exists
- `400` - Phone number already in use
- `400` - Invalid verification code
- `400` - Password too short

---

7. Quick Reference

7.1 Authentication Flow Summary

1. **Register**: `POST /auth/register` → Send OTP
2. **Verify**: `POST /auth/verify-phone` → Create account
3. **Login**: `POST /auth/login` → Get JWT tokens
4. **Access**: Use `Authorization: Bearer {{token}}` header
5. **Refresh**: `POST /auth/refresh` → Rotate tokens
6. **Logout**: `POST /auth/logout` → Revoke session

7.2 Role-Based Access

Role	Permissions
BUYER	Profile management, order placement
VENDOR	Product management, order fulfillment
ADMIN	Vendor management, user administration
SUPER_ADMIN	All permissions including role management

7.3 Token Management

- **Access Token**: 1 hour expiration, used for API calls
- **Refresh Token**: 30 days expiration, stored securely
- **Invite Token**: 14 days expiration, vendor registration only

---

8. Development Notes

8.1 Environment Setup

Required Environment Variables:

```
DATABASE_URL=postgresql://...
JWT_ACCESS_SECRET=your-access-secret
JWT_REFRESH_SECRET=your-refresh-secret
```

```
JWT_INVITE_SECRET=your-invite-secret
TWILIO_ACCOUNT_SID=your-twilio-sid
TWILIO_AUTH_TOKEN=your-twilio-token
TWILIO_PHONE_NUMBER=+1234567890
CLIENT_URL=http://localhost:3000
```

8.2 Testing Best Practices

- Use unique email/phone for each test
- Clear environment variables between test runs
- Test both success and error scenarios
- Verify response schemas match documentation
- Use Postman collections for organized testing

8.3 Security Considerations

- Always use HTTPS in production
- Implement rate limiting on auth endpoints
- Validate all input data
- Use secure password hashing
- Implement proper session management
- Log security events appropriately

---

9. Support & Troubleshooting

9.1 Common Issues

OTP Not Received:

- Check phone number format (E.164)
- Verify Twilio configuration
- Check SMS delivery logs

Login Failed:

- Verify email is confirmed
- Check user status is ACTIVE
- Ensure password is correct

Vendor Registration Issues:

- Verify invite token is valid
- Check admin approval status

- Ensure all required fields are provided

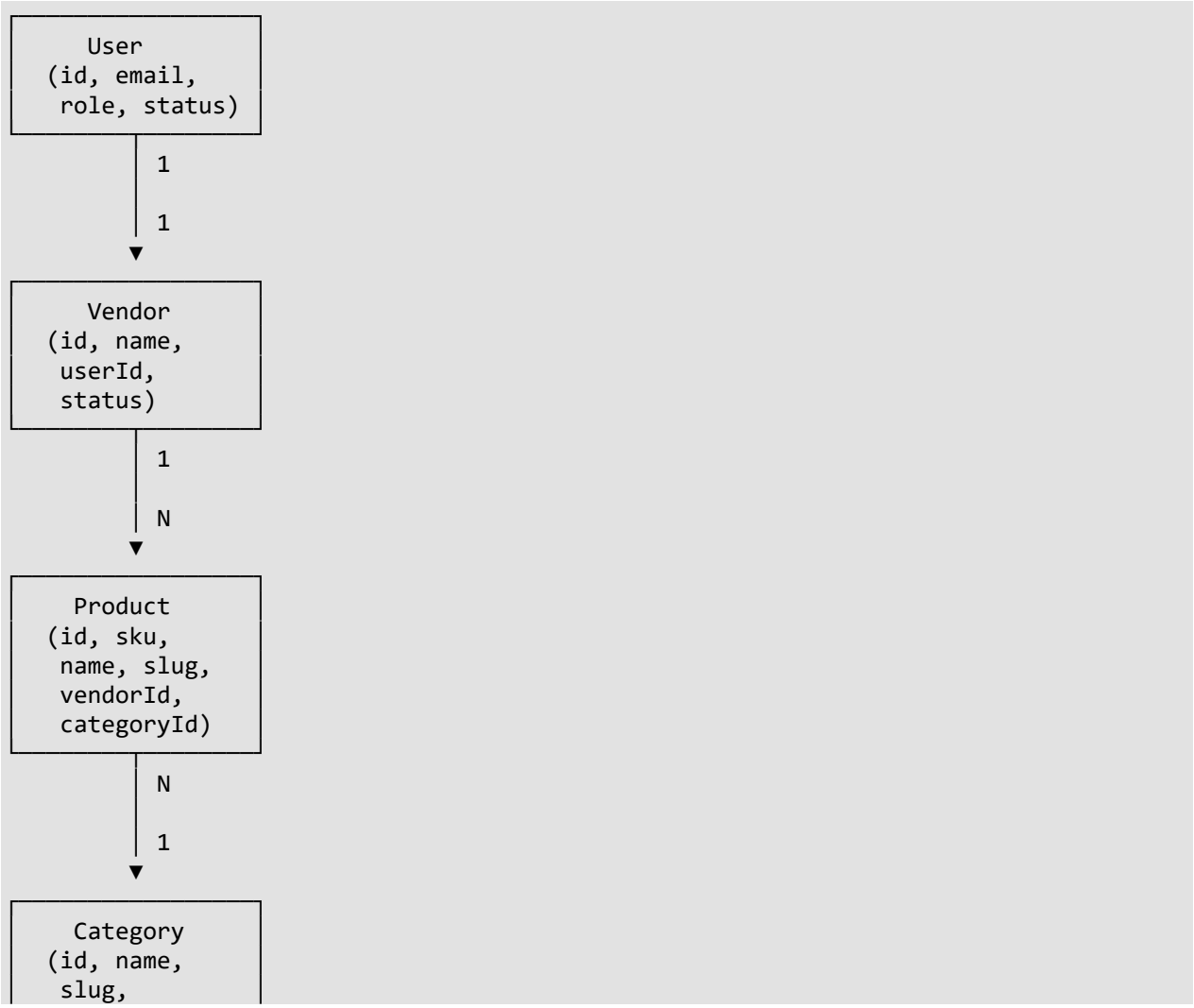
9.2 Debugging Tips

1. Check API response bodies for detailed error messages
2. Verify request headers include proper authentication
3. Ensure environment variables are correctly set
4. Check network connectivity and CORS settings
5. Review server logs for detailed error information

Product Module API Documentation

This comprehensive API documentation provides detailed specifications for the Product Module, designed for project managers, frontend developers, and QA teams. Each endpoint includes implementation status, testing procedures, and integration guidelines

Data Relationships



```
    parentId)
```

User Roles & Functionality

Buyer Functionality Public Access

Role Description: Buyers are public users who can browse and search products without authentication. They have read-only access to product data.

Available Features

Feature	Status	Description
Category Browsing		Browse hierarchical category tree
Product Search		Advanced search with filters
Product Details		View complete product information
Price Filtering		Filter by price range
Stock Filtering		Filter by stock status

Featured Products		View featured products only

Key Endpoints

- `GET /api/v1/categories` - Get category tree
- `GET /api/v1/products` - Search and filter products
- `GET /api/v1/products/:id` - Get product details

Data Access Permissions

- **Can View**: Active products only
- **Cannot View**: Draft, inactive, or soft-deleted products
- **Cannot Modify**: Any product data

Business Use Cases

- Product discovery and browsing
- Price comparison across vendors
- Category-based navigation
- Finding specific products via search

Vendor Functionality

Role Description: Vendors are authenticated users who can create, manage, and sell their own products. Each vendor has a unique vendor profile linked to their user account.

Available Features

Feature	Status	Description
Product Creation		Create new products

Product Updates		Update own products
Product Deletion		Soft delete products
Product Restoration		Restore deleted products
Image Upload		Upload product images
Inventory Management		Manage stock levels
Pricing Control		Set prices and discounts
Status Management		Control product visibility

Key Endpoints

- `POST /api/v1/products` - Create product

- `PUT /api/v1/products/:id` - Update own product
- `DELETE /api/v1/products/:id` - Delete own product
- `POST /api/v1/products/:id/restore` - Restore own product
- `POST /api/v1/products/:id/images` - Upload images

Security Measures

- **Authentication**: JWT token required
- **Authorization**: Vendor role validation
- **Ownership**: Vendor ID mapped from user ID
- **Verification**: Ownership check before operations

Business Use Cases

- Product catalog management
- Inventory control and tracking
- Price management and promotions
- Product marketing (featured status)
- Visual content management

Admin Functionality

Role Description: Admins have full system access to manage all products across all vendors. They can oversee and control the entire product ecosystem.

Available Features

Feature	Status	Description
Admin Product Creation		Create for any vendor

Global Product Updates		Update any product
System-wide Deletion		Delete any product
Product Restoration		Restore any product
Draft Product Access		View all product states
Vendor Monitoring		Monitor vendor catalogs
Bulk Operations		Batch product management
Category Management		Admin category CRUD

Key Endpoints

- `POST /api/admin/v1/products` - Create product for any vendor

- `PUT /api/admin/v1/products/:id` - Update any product
- `DELETE /api/admin/v1/products/:id` - Delete any product
- `PATCH /api/admin/v1/products/:id/restore` - Restore any product
- `GET /api/admin/v1/products` - Search all products
- `GET /api/admin/v1/products/vendor/:vendorId` - Get vendor products

Security Measures

- **Authentication**: JWT token required
- **Authorization**: ADMIN or SUPER_ADMIN role
- **Elevated Permissions**: Override vendor restrictions
- **Audit Trail**: Operation logging considerations

Business Use Cases

- Product quality control
- Vendor support and assistance
- System-wide product management
- Content moderation
- Analytics and reporting
- Compliance enforcement

Authentication & Security

Authentication Flow

The Product Module uses JWT (JSON Web Token) based authentication with role-based authorization.

Token Acquisition

```
POST /api/v1/auth/login
Content-Type: application/json

{
  "email": "user@example.com",
  "password": "password123"
}
```

Token Usage

```
Authorization: Bearer <jwt_token>
```

Token Structure

```
{
  "sub": "user-uuid",
  "email": "user@example.com",
  "role": "VENDOR|ADMIN|SUPER_ADMIN",
  "iat": 1640995200,
  "exp": 1641081600
}
```

Role-Based Access Control

Role	Access Level	Permissions
Public	Read-only	Browse products, search, view categories
Vendor	Own products	CRUD own products, upload images
Admin	System-wide	Full access to all products and vendors
Super Admin	Complete	All admin permissions + system configuration

Security Measures

Authentication Security

- **JWT Secret**: 256-bit secret key
- **Token Expiry**: 24 hours (configurable)
- **Refresh Tokens**: Supported for extended sessions
- **Password Hashing**: bcrypt with salt rounds

Authorization Security

- **Role Validation**: Middleware-based role checking
- **Resource Ownership**: Vendor product ownership verification
- **Route Protection**: Endpoint-level authorization
- **Permission Scoping**: Granular permission control

Data Security

- **Input Validation**: Zod schema validation
- **SQL Injection Prevention**: Prisma ORM parameterization
- **XSS Protection**: Express security headers
- **Rate Limiting**: Request throttling (configurable)

API Security

- **CORS Configuration**: Cross-origin resource sharing control
- **HTTPS Enforcement**: SSL/TLS required in production
- **API Versioning**: Versioned endpoint structure
- **Audit Logging**: Operation tracking (planned)

Public Endpoints

Base URL: `http://localhost:3000/api/v1`

1. Get Category Tree

Endpoint: `GET /api/v1/categories`

Description: Retrieves the complete category hierarchy as a nested tree structure. Each category includes its product count (recursive count including all subcategories). Results are cached for 1 hour for optimal performance.

Authentication: Not required (Public)

Rate Limiting: 100 requests per minute

Cache TTL: 1 hour

Request Details

Attribute	Value
Method	GET
URL	`/api/v1/categories`
Headers	None required
Body	Not applicable

Response Schema

```
{
  "success": true,
```

```
  "data": [
    {
      "id": "uuid",
      "name": "Electronics",
      "slug": "electronics",
      "imageUrl": "https://example.com/image.jpg",
      "productCount": 150,
      "children": [
        {
          "id": "uuid",
          "name": "Computers",
          "slug": "computers",
          "imageUrl": null,
          "productCount": 80,
          "children": []
        }
      ]
    }
  ]
}
```

Postman Testing Guide

Step 1: Create Request

1. Open Postman
2. Click **New** > **Request**
3. Set method to **GET**
4. Enter URL: `http://localhost:3000/api/v1/categories`

Step 2: Send Request

1. Click **Send**
2. Verify response status: `200 OK`
3. Check response structure matches schema above

Expected Response Time

- **Cached Response**: < 50ms
- **Fresh Response**: < 200ms

Error Scenarios

Error	Status Code	Description
Database Error	500	Internal server error
Cache Error	500	Redis connection failed

2. Search Products

Endpoint: `GET /api/v1/products`

Description: Search and filter products with advanced filtering options. Supports full-text search across name, SKU, description, and tags. Results are cached for 5 minutes.

Authentication: Not required (Public)

Rate Limiting: 100 requests per minute

Cache TTL: 5 minutes

Query Parameters

Parameter	Type	Default	Required	Description
`q`	string	-	No	Search term (ILIKE on name, sku, description, tags)
`vendorId`	string	-	No	Comma-separated vendor IDs
`categoryId`	string	-	No	Filter by category (includes subcategories)
`priceMin`	number	-	No	Minimum basePrice
`priceMax`	number	-	No	Maximum basePrice
`stockStatus`	enum	-	No	IN_STOCK, LOW_STOCK, OUT_OF_STOCK, BACKORDER
`isFeatured`	boolean	-	No	Featured products only
`status`	enum	ACTIVE	No	Product status filter
`sortBy`	string	createdAt	No	createdAt, basePrice, name
`sortOrder`	string	desc	No	asc, desc
`page`	number	1	No	Page number
`limit`	number	20	No	Items per page (max 100)

Request Details

Attribute	Value
Method	GET
URL	`/api/v1/products`
Headers	None required

Query Params	See table above

Response Schema


```
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "sku": "PROD-001",
      "name": "Laptop Pro",
      "slug": "laptop-pro",
      "description": "High-performance laptop",
      "basePrice": 999.99,
      "salePrice": 899.99,
      "unit": "each",
      "minOrderQty": 1,
      "maxOrderQty": 10,
      "stockQty": 50,
      "stockStatus": "IN_STOCK",
      "images": ["https://example.com/image.jpg"],
      "attributes": {},
      "tags": ["laptop", "computer"],
      "status": "ACTIVE",
      "isFeatured": true,
      "vendor": {
        "id": "uuid",
        "name": "TechStore",
        "slug": "techstore"
      },
      "category": {
        "id": "uuid",
        "name": "Laptops",
        "slug": "laptops"
      },
      "createdAt": "2024-01-01T00:00:00.000Z",
      "updatedAt": "2024-01-01T00:00:00.000Z"
    }
  ],
  "meta": {
    "currentPage": 1,
    "totalPages": 5,
    "totalItems": 100,
    "itemsPerPage": 20,
    "hasNextPage": true,
    "hasPrevPage": false
  }
}
```


Postman Testing Guide

Step 1: Basic Search

1. Create GET request: `http://localhost:3000/api/v1/products`

2. Add query parameters:
    - `q`: `laptop`
    - `priceMin`: `500`
    - `priceMax`: `1500`
    - `page`: `1`
    - `limit`: `10`
3. Click **Send**
4. Verify status: `200 OK`

Step 2: Advanced Filtering

1. Modify query parameters:
    - `categoryId`: `{valid-category-uuid}`
    - `vendorId`: `{valid-vendor-uuid}`
    - `stockStatus`: `IN_STOCK`
    - `isFeatured`: `true`
    - `sortBy`: `basePrice`
    - `sortOrder`: `asc`

Expected Response Time

- **Cached Response**: < 100ms
- **Fresh Response**: < 500ms

Error Scenarios

Error	Status Code	Description
Invalid Parameters	400	Malformed query parameters
Database Error	500	Internal server error

---

3. Get Product by ID 

Endpoint: `GET /api/v1/products/:id`

Description: Retrieve a single product by its ID with full details including vendor and category information.

Authentication: Not required (Public)

Rate Limiting: 200 requests per minute

Cache TTL: 30 minutes

Request Details

Attribute	Value
Method	GET
URL	`/api/v1/products/:id`
Path Param	`id` (string, required) - Product UUID
Headers	None required

Response Schema


```
{
  "success": true,
  "data": {
    "id": "uuid",
    "sku": "PROD-001",
    "name": "Laptop Pro",
    "slug": "laptop-pro",
    "description": "High-performance laptop",
    "basePrice": 999.99,
    "salePrice": 899.99,
    "unit": "each",
    "minOrderQty": 1,
    "maxOrderQty": 10,
    "stockQty": 50,
    "stockStatus": "IN_STOCK",
    "images": ["https://example.com/image.jpg"],
    "attributes": {},
    "tags": ["laptop", "computer"],
    "status": "ACTIVE",
    "isFeatured": true,
    "vendor": {
      "id": "uuid",
      "name": "TechStore",
      "slug": "techstore"
    },
    "category": {
      "id": "uuid",
      "name": "Laptops",
      "slug": "laptops"
    },
    "createdAt": "2024-01-01T00:00:00.000Z",
    "updatedAt": "2024-01-01T00:00:00.000Z"
```

```
  }
}
```

Expected Response Time

- **Cached Response**: < 50ms
- **Fresh Response**: < 200ms

Error Scenarios

Error	Status Code	Description
Product Not Found	404	Invalid product UUID
Invalid UUID Format	400	Malformed UUID parameter

Vendor Endpoints 

Base URL: `http://localhost:3000/api/v1`
Authentication: Required (Vendor role)
Rate Limiting: 60 requests per minute

1. Create Product 

Endpoint: `POST /api/v1/products`

Description: Create a new product. Only authenticated vendors can create products. The vendor ID is automatically set from the authenticated user. SKU must be unique across all products. Slug is auto-generated from the product name.

Request Details

Attribute	Value
Method	POST
URL	`/api/v1/products`
Headers	`Authorization: Bearer <jwt_token>`
Content-Type	`application/json`

Request Body Schema

```
{
  "sku": "PROD-001",
  "name": "Laptop Pro",
  "description": "High-performance laptop for professionals",
  "basePrice": 999.99,
  "salePrice": 899.99,
  "unit": "each",
  "minOrderQty": 1,
  "maxOrderQty": 10,
  "stockQty": 50,
  "categoryId": "uuid",
  "tags": ["laptop", "computer", "pro"],
  "attributes": {
    "brand": "TechBrand",
    "warranty": "2 years"
  },
  "status": "DRAFT",
  "isFeatured": false
}
```

Validation Rules

Field	Required	Validation
`sku`	Yes	Unique string, min 1 character
`name`	Yes	String, min 2 characters
`basePrice`	Yes	Number > 0
`salePrice`	No	Must be < basePrice if provided
`unit`	Yes	String, min 1 character
`stockQty`	No	Integer >= 0
`minOrderQty`	No	Integer >= 1, default 1
`maxOrderQty`	No	Integer >= minOrderQty
`categoryId`	No	Valid category UUID
`status`	No	Enum: ACTIVE, INACTIVE, DRAFT, DISCONTINUED

Response Schema

```
{
  "success": true,
  "data": {
    "id": "uuid",
    "sku": "PROD-001",
    "name": "Laptop Pro",
    "slug": "laptop-pro",
    "description": "High-performance laptop for professionals",
```

```
    "basePrice": 999.99,
    "salePrice": 899.99,
    "unit": "each",
    "minOrderQty": 1,
    "maxOrderQty": 10,
    "stockQty": 50,
    "stockStatus": "IN_STOCK",
    "images": [],
    "attributes": {
      "brand": "TechBrand",
      "warranty": "2 years"
    },
    "tags": ["laptop", "computer", "pro"],
    "status": "DRAFT",
    "isFeatured": false,
    "vendor": {
      "id": "uuid",
      "name": "TechStore",
      "slug": "techstore"
    },
    "category": {
      "id": "uuid",
      "name": "Laptops",
      "slug": "laptops"
    },
    "createdAt": "2024-01-01T00:00:00.000Z",
    "updatedAt": "2024-01-01T00:00:00.000Z"
  }
}
```

Postman Testing Guide

Step 1: Authenticate as Vendor

1. Create POST request: `http://localhost:3000/api/v1/auth/login`
2. Body: `{ "email": "vendor@example.com", "password": "password123" }`
3. Copy the `token` from response

Step 2: Create Product

1. Create POST request: `http://localhost:3000/api/v1/products`
2. **Headers** tab:
    - Key: `Authorization`
    - Value: `Bearer {your_token}`
    - Key: `Content-Type`
    - Value: `application/json`
3. **Body** tab:
    - Select **raw** and **JSON**

- Paste the request body JSON

4. Click **Send**
5. Verify status: `201 Created`

Expected Response Time

- **Average**: < 500ms
- **With Image Upload**: < 2s

Error Scenarios

Error	Status Code	Description
Missing Required Fields	400	Validation errors in request body
Duplicate SKU	400	SKU already exists in system
Invalid Category	400	Category UUID not found
Not Authenticated	401	Missing or invalid JWT token
Not Vendor	403	User does not have VENDOR role

2. Update Product

Endpoint: `PUT /api/v1/products/:id`

Description: Update an existing product. Vendors can only update their own products. If status changes to ACTIVE, `publishedAt` is automatically set to current date. Slug is regenerated if name changes.

Request Details

Attribute	Value
Method	PUT
URL	`/api/v1/products/:id`
Path Param	`id` (string, required) - Product UUID
Headers	`Authorization: Bearer <jwt_token>`
Content-Type	`application/json`

Request Body Schema

```
{
  "name": "Laptop Pro Updated",
  "description": "Updated description",
  "basePrice": 1099.99,
  "salePrice": 999.99,
  "unit": "each",
  "minOrderQty": 1,
  "maxOrderQty": 15,
  "stockQty": 45,
  "categoryId": "uuid",
  "status": "ACTIVE",
  "isFeatured": true
}
```

Response Schema

```
{
  "success": true,
  "data": {
    "id": "uuid",
    "sku": "PROD-001",
    "name": "Laptop Pro Updated",
    "slug": "laptop-pro-updated",
    "description": "Updated description",
    "basePrice": 1099.99,
    "salePrice": 999.99,
    "unit": "each",
    "minOrderQty": 1,
    "maxOrderQty": 15,
    "stockQty": 45,
    "stockStatus": "IN_STOCK",
    "images": [],
    "attributes": {},
    "tags": [],
    "status": "ACTIVE",
    "isFeatured": true,
    "vendor": {
      "id": "uuid",
      "name": "TechStore",
      "slug": "techstore"
    },
    "category": {
      "id": "uuid",
      "name": "Laptops",
      "slug": "laptops"
    },
    "createdAt": "2024-01-01T00:00:00.000Z",
    "updatedAt": "2024-01-02T00:00:00.000Z",
    "publishedAt": "2024-01-02T00:00:00.000Z"
  }
}
```

Postman Testing Guide

Step 1: Update Product

1. Use vendor token from previous authentication
2. Create PUT request: `http://localhost:3000/api/v1/products/{product_id}`
3. Add Authorization header with Bearer token
4. In **Body** tab (raw JSON), paste the request body
5. Click **Send**
6. Verify status: `200 OK`

Error Scenarios

Error	Status Code	Description
Product Not Found	404	Invalid product UUID
Not Product Owner	403	"Unauthorized: You can only update your own products"
Slug Conflict	400	"Product slug conflict"

3. Delete Product

Endpoint: `DELETE /api/v1/products/:id`

Description: Soft delete a product by setting `deletedAt` timestamp. Deleted products are excluded from all search queries by default. Vendors can only delete their own products.

Request Details

Attribute	Value
Method	DELETE
URL	`/api/v1/products/:id`
Path Param	`id` (string, required) - Product UUID
Headers	`Authorization: Bearer <jwt_token>`

Response Schema

```
{
  "success": true,
```

```
  "message": "Product deleted successfully"
}
```

Postman Testing Guide

Step 1: Delete Product

1. Create DELETE request: `http://localhost:3000/api/v1/products/{product_id}`
2. Add Authorization header with Bearer token
3. Click **Send**
4. Verify status: `200 OK`

Step 2: Verification

1. Try to get the product via GET endpoint: `GET /api/v1/products/{product_id}`
2. Should return `404 Not Found`
3. Try to search products: `GET /api/v1/products`
4. Deleted product should not appear in results

Error Scenarios

Error	Status Code	Description
Product Not Found	404	Invalid product UUID
Not Product Owner	403	"Unauthorized: You can only delete your own products"

4. Restore Product

Endpoint: `POST /api/v1/products/:id/restore`

Description: Restore a soft-deleted product by setting `deletedAt` to null. Vendors can only restore their own products.

Request Details

Attribute	Value
Method	POST
URL	`/api/v1/products/:id/restore`
Path Param	`id` (string, required) - Product UUID

Headers	`Authorization: Bearer <jwt_token>`

Response Schema


```
{
  "success": true,
  "data": {
    "id": "uuid",
    "sku": "PROD-001",
    "name": "Laptop Pro Updated",
    "slug": "laptop-pro-updated",
    "description": "Updated description",
    "basePrice": 1099.99,
    "salePrice": 999.99,
    "unit": "each",
    "minOrderQty": 1,
    "maxOrderQty": 15,
    "stockQty": 45,
    "stockStatus": "IN_STOCK",
    "images": [],
    "attributes": {},
    "tags": [],
    "status": "ACTIVE",
    "isFeatured": true,
    "vendor": {
      "id": "uuid",
      "name": "TechStore",
      "slug": "techstore"
    },
    "category": {
      "id": "uuid",
      "name": "Laptops",
      "slug": "laptops"
    },
    "createdAt": "2024-01-01T00:00:00.000Z",
    "updatedAt": "2024-01-02T00:00:00.000Z"
  }
}
```


Postman Testing Guide

Step 1: Restore Product

1. Create POST request: `http://localhost:3000/api/v1/products/{product_id}/restore`
2. Add Authorization header with Bearer token
3. Click **Send**
4. Verify status: `200 OK`

Step 2: Verification

1. Try to get the product via GET endpoint: `GET /api/v1/products/{product_id}`
2. Should return product data with `200 OK`

Error Scenarios

Error	Status Code	Description
Product Not Found	404	Invalid product UUID
Not Product Owner	403	"Unauthorized: You can only restore your own products"

5. Upload Product Images

Endpoint: `POST /api/v1/products/:id/images`

Description: Upload multiple images for a product. Images are stored on Cloudinary and URLs are appended to the product's images array. Maximum 10 images per request, 5MB each. Supported formats: JPG, PNG, WebP.

Request Details

Attribute	Value
Method	POST
URL	`/api/v1/products/:id/images`
Path Param	`id` (string, required) - Product UUID
Headers	`Authorization: Bearer <jwt_token>`
Body Type	form-data

Request Body

- **Key**: `images` (type: File)
- **Value**: Select one or more image files (max 10)
- **File Size Limit**: 5MB per file
- **Supported Formats**: JPG, PNG, WebP

Response Schema

```
{
  "success": true,
  "data": {
    "id": "uuid",
    "sku": "PROD-001",
    "name": "Laptop Pro Updated",
    "slug": "laptop-pro-updated",
```

```
    "description": "Updated description",
    "basePrice": 1099.99,
    "salePrice": 999.99,
    "unit": "each",
    "minOrderQty": 1,
    "maxOrderQty": 15,
    "stockQty": 45,
    "stockStatus": "IN_STOCK",
    "images": [
      "https://res.cloudinary.com/your-cloud/image1.jpg",
      "https://res.cloudinary.com/your-cloud/image2.jpg"
    ],
    "attributes": {},
    "tags": [],
    "status": "ACTIVE",
    "isFeatured": true,
    "vendor": {
      "id": "uuid",
      "name": "TechStore",
      "slug": "techstore"
    },
    "category": {
      "id": "uuid",
      "name": "Laptops",
      "slug": "laptops"
    },
    "createdAt": "2024-01-01T00:00:00.000Z",
    "updatedAt": "2024-01-02T00:00:00.000Z"
  }
}
```

Postman Testing Guide

Step 1: Upload Images

1. Create POST request: `http://localhost:3000/api/v1/products/{product_id}/images`
2. Add Authorization header with Bearer token
3. Go to **Body** tab
4. Select **form-data**
5. Add key: `images` (type: File)
6. Select one or more image files from your computer
7. Click **Send**
8. Verify status: `200 OK`

Expected Response Time

- **Average**: < 2s per image
- **Multiple Images**: < 5s for up to 10 images

Error Scenarios

Error	Status Code	Description
Product Not Found	404	Invalid product UUID
Not Product Owner	403	"Unauthorized: You can only upload images to your own products"
File Too Large	400	"File size exceeds 5MB limit"
Invalid File Type	400	"Unsupported file format"
Cloudinary Error	500	"Image upload failed"

Admin Endpoints

Base URL: `http://localhost:3000/api/admin/v1/products`
Authentication: Required (ADMIN or SUPER_ADMIN role)
Rate Limiting: 100 requests per minute

1. Create Product (Admin)

Endpoint: `POST /api/admin/v1/products`

Description: Create a new product as an admin. Admins can specify the vendor ID to create products on behalf of vendors. All other functionality is the same as vendor product creation.

Request Details

Attribute	Value
Method	POST
URL	`/api/admin/v1/products`
Headers	`Authorization: Bearer <admin_jwt_token>`
Content-Type	`application/json`

Request Body Schema

```
{
  "sku": "ADMIN-PROD-001",
  "name": "Admin Laptop",
```

```
  "description": "Product created by admin",
  "basePrice": 1299.99,
  "salePrice": 1199.99,
  "unit": "each",
  "minOrderQty": 1,
  "maxOrderQty": 20,
  "stockQty": 100,
  "vendorId": "vendor-uuid",
  "categoryId": "category-uuid",
  "tags": ["admin", "laptop"],
  "status": "ACTIVE",
  "isFeatured": true
}
```

Additional Field (Admin only)

Field	Required	Description
`vendorId`	Yes	The vendor ID to assign the product to

Response Schema

Same as vendor create product response

Postman Testing Guide

Step 1: Authenticate as Admin

1. Create POST request: `http://localhost:3000/api/v1/auth/login`
2. Body: `{ "email": "admin@example.com", "password": "admin123" }`
3. Copy the `token` from response

Step 2: Create Product as Admin

1. Create POST request: `http://localhost:3000/api/admin/v1/products`
2. Add Authorization header with admin token
3. Include `vendorId` in request body
4. Click **Send**
5. Verify status: `201 Created`

Error Scenarios

Error	Status Code	Description
Missing vendorId	400	"Vendor ID is required"
Invalid vendorId	400	"Vendor not found"
Not Admin	403	User does not have admin role

2. Update Product (Admin)

Endpoint: `PUT /api/admin/v1/products/:id`

Description: Update any product in the system. Admins can update products belonging to any vendor.

Request Details

Attribute	Value
Method	PUT
URL	`/api/admin/v1/products/:id`
Path Param	`id` (string, required) - Product UUID
Headers	`Authorization: Bearer <admin_jwt_token>`

Response Schema

Same as vendor update product response

Postman Testing Guide

Step 1: Update Any Product

1. Use admin token
2. Create PUT request: `http://localhost:3000/api/admin/v1/products/{any_product_id}`
3. Add Authorization header
4. Send update data
5. Verify status: `200 OK`

Error Scenarios

Error	Status Code	Description
Product Not Found	404	Invalid product UUID
Not Admin	403	User does not have admin role

3. Delete Product (Admin)

Endpoint: `DELETE /api/admin/v1/products/:id`

Description: Soft delete any product in the system. Admins can delete products belonging to any vendor.

Request Details

Attribute	Value
Method	DELETE
URL	`/api/admin/v1/products/:id`
Path Param	`id` (string, required) - Product UUID
Headers	`Authorization: Bearer <admin_jwt_token>`

Response Schema

```
{
  "success": true,
  "message": "Product deleted successfully"
}
```

Postman Testing Guide

Step 1: Delete Any Product

1. Create DELETE request: `http://localhost:3000/api/admin/v1/products/{product_id}`
2. Add admin Authorization header
3. Click **Send**
4. Verify status: `200 OK`

4. Restore Product (Admin)

Endpoint: `PATCH /api/admin/v1/products/:id/restore`

Description: Restore any soft-deleted product in the system.

Request Details

Attribute	Value
Method	PATCH
URL	`/api/admin/v1/products/:id/restore`
Path Param	`id` (string, required) - Product UUID
Headers	`Authorization: Bearer <admin_jwt_token>`

Response Schema

Same as vendor restore product response

5. Search All Products (Admin)

Endpoint: `GET /api/admin/v1/products`

Description: Search and filter all products in the system, including draft and inactive products. Admins can view all product states.

Query Parameters

Same as public search but with additional access to all product statuses.

Request Details

Attribute	Value
Method	GET
URL	`/api/admin/v1/products`
Headers	`Authorization: Bearer <admin_jwt_token>`

Response Schema

Same as public search but includes products with all statuses (DRAFT, INACTIVE, etc.)

Postman Testing Guide

Step 1: Search All Products

1. Create GET request: `http://localhost:3000/api/admin/v1/products`
2. Add admin Authorization header

3. Add query parameters as needed
4. Click **Send**
5. Verify status: `200 OK`

Step 2: Search Draft Products

1. Add query parameter: `status=DRAFT`
2. Should return draft products not visible to public

6. Get Vendor Products (Admin)

Endpoint: `GET /api/admin/v1/products/vendor/:vendorId`

Description: Retrieve all products for a specific vendor. Useful for vendor monitoring and support.

Request Details

Attribute	Value
Method	GET
URL	`/api/admin/v1/products/vendor/:vendorId`
Path Param	`vendorId` (string, required) - Vendor UUID
Headers	`Authorization: Bearer <admin_jwt_token>`

Response Schema

```
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "sku": "PROD-001",
      "name": "Product Name",
      "status": "ACTIVE",
      "vendor": {
        "id": "vendor-uuid",
        "name": "Vendor Name"
      }
      // ... other product fields
    }
  ],
  "meta": {
    "totalItems": 25,
```

```
    "currentPage": 1,
    "totalPages": 3
  }
}
```

Postman Testing Guide

Step 1: Get Vendor Products

1. Create GET request: `http://localhost:3000/api/admin/v1/products/vendor/{vendor_id}`
2. Add admin Authorization header
3. Click **Send**
4. Verify status: `200 OK`

Error Scenarios

Error	Status Code	Description
Vendor Not Found	404	Invalid vendor UUID
Not Admin	403	User does not have admin role

Testing Guide

Comprehensive Testing Strategy

1. Environment Setup

```
# Required Environment Variables
NODE_ENV=development
PORT=3000
DATABASE_URL="postgresql://..."
JWT_ACCESS_SECRET="your-secret"
REDIS_URL="redis://localhost:6379"
CLOUDINARY_URL="cloudinary://..."
```

2. Test Data Preparation

```
# Create test users and vendors
npm run seed:test-data

# Expected test data:
- 1 Super Admin user
- 2 Admin users
- 3 Vendor users
```

```
- 5 Categories
- 20 Sample products
```

3. Postman Collection Setup

Environment Variables:

```
baseUrl: http://localhost:3000
vendorEmail: vendor@example.com
vendorPassword: password123
adminEmail: admin@example.com
adminPassword: admin123
vendorToken: {{vendorToken}}
adminToken: {{adminToken}}
```

4. Test Scenarios Matrix

Scenario	Method	Endpoint	Auth Required	Expected Status
Public Category Browse	GET	`/api/v1/categories`	No	200
Public Product Search	GET	`/api/v1/products`	No	200
Public Product Details	GET	`/api/v1/products/:id`	No	200
Vendor Product Create	POST	`/api/v1/products`	Vendor	201
Vendor Product Update	PUT	`/api/v1/products/:id`	Vendor	200
Vendor Product Delete	DELETE	`/api/v1/products/:id`	Vendor	200
Admin Product Create	POST	`/api/admin/v1/products`	Admin	201
Admin Global Search	GET	`/api/admin/v1/products`	Admin	200
Admin Vendor Products	GET	`/api/admin/v1/products/vendor/:id`	Admin	200

5. Performance Testing

Response Time Benchmarks:

- **Cached Responses**: < 100ms
- **Database Queries**: < 500ms

- **File Uploads**: < 2s per image
- **Complex Searches**: < 1s

Load Testing Recommendations:

```
# Using Apache Bench
ab -n 1000 -c 10 http://localhost:3000/api/v1/categories

# Using Artillery (recommended)
artillery run load-test-config.yml
```

6. Error Handling Tests

Common Error Scenarios:

```
// Test 401 Unauthorized
pm.test("Returns 401 when not authenticated", function () {
    pm.response.to.have.status(401);
});

// Test 403 Forbidden
pm.test("Returns 403 when wrong role", function () {
    pm.response.to.have.status(403);
});

// Test 404 Not Found
pm.test("Returns 404 for invalid resource", function () {
    pm.response.to.have.status(404);
});

// Test 400 Validation Error
pm.test("Returns 400 for invalid data", function () {
    pm.response.to.have.status(400);
    const jsonData = pm.response.json();
    pm.expect(jsonData).to.have.property('errors');
});
```

7. Integration Testing Checklist

- [ ] Authentication flow works for all roles
- [ ] Authorization prevents cross-vendor access
- [ ] Public endpoints work without authentication
- [ ] Admin endpoints override vendor restrictions
- [ ] Soft delete prevents public access
- [ ] Cache invalidation works on updates
- [ ] Image upload stores files correctly
- [ ] Search filters work as expected
- [ ] Pagination works correctly

- [ ] Rate limiting is enforced

2. Set method to `GET`
3. Enter URL: `http://localhost:3000/api/v1/categories`
4. Click **Send**
5. Verify response contains nested category tree with product counts
6. Expected status: `200 OK`

---

2. Search Products

Endpoint: `GET /api/v1/products`

Description: Search and filter products with advanced filtering options. Supports full-text search across name, SKU, description, and tags. Results are cached for 5 minutes.

Authentication: Not required (Public)

Query Parameters:

Parameter	Type	Default	Description
q	string	-	Search term (ILIKE on name, sku, description, tags)
vendorId	string	-	Comma-separated vendor IDs
categoryId	string	-	Filter by category (includes subcategories)
priceMin	number	-	Minimum basePrice
priceMax	number	-	Maximum basePrice
stockStatus	enum	-	IN_STOCK, LOW_STOCK, OUT_OF_STOCK, BACKORDER
isFeatured	boolean	-	Featured products only
status	enum	ACTIVE	Product status filter
sortBy	string	createdAt	createdAt, basePrice, name
sortOrder	string	desc	asc, desc
page	number	1	Page number
limit	number	20	Items per page (max 100)

Response:

```
{
  "success": true,
  "data": [
    {
      "id": "uuid",
      "sku": "PROD-001",
```

```
      "name": "Laptop Pro",
      "slug": "laptop-pro",
      "description": "High-performance laptop",
      "basePrice": 999.99,
      "salePrice": 899.99,
      "unit": "each",
      "minOrderQty": 1,
      "maxOrderQty": 10,
      "stockQty": 50,
      "stockStatus": "IN_STOCK",
      "images": ["https://example.com/image.jpg"],
      "attributes": {},
      "tags": ["laptop", "computer"],
      "status": "ACTIVE",
      "isFeatured": true,
      "vendor": {
        "id": "uuid",
        "name": "TechStore",
        "slug": "techstore"
      },
      "category": {
        "id": "uuid",
        "name": "Laptops",
        "slug": "laptops"
      },
      "createdAt": "2024-01-01T00:00:00.000Z",
      "updatedAt": "2024-01-01T00:00:00.000Z"
    }
  ],
  "meta": {
    "currentPage": 1,
    "totalPages": 5,
    "totalItems": 100,
    "itemsPerPage": 20,
    "hasNextPage": true,
    "hasPrevPage": false
  }
}
```


Postman Testing Steps:

1. Create a new GET request
2. Enter URL: `http://localhost:3000/api/v1/products`
3. Add query parameters in **Params** tab:
    - `q`: `laptop`
    - `priceMin`: `500`
    - `priceMax`: `1500`
    - `stockStatus`: `IN_STOCK`
    - `page`: `1`

- `limit`: `10`

4. Click **Send**
5. Verify filtered results match criteria
6. Expected status: `200 OK`

Test Different Scenarios:

- Search by category: Add `categoryId` with a valid category UUID
- Search by vendor: Add `vendorId` with comma-separated vendor UUIDs
- Sort by price: Add `sortBy=basePrice&sortOrder=asc`
- Featured only: Add `isFeatured=true`

---

3. Get Product by ID

Endpoint: `GET /api/v1/products/:id`

Description: Retrieve a single product by its ID with full details including vendor and category information.

Authentication: Not required (Public)

Path Parameters:

- `id` (string, required): Product UUID

Response:


```
{
  "success": true,
  "data": {
    "id": "uuid",
    "sku": "PROD-001",
    "name": "Laptop Pro",
    "slug": "laptop-pro",
    "description": "High-performance laptop",
    "basePrice": 999.99,
    "salePrice": 899.99,
    "unit": "each",
    "minOrderQty": 1,
    "maxOrderQty": 10,
    "stockQty": 50,
    "stockStatus": "IN_STOCK",
    "images": ["https://example.com/image.jpg"],
    "attributes": {},
    "tags": ["laptop", "computer"],
    "status": "ACTIVE",
    "isFeatured": true,
    "vendor": {
      "id": "uuid",
      "name": "TechStore",
```

```
      "slug": "techstore"
    },
    "category": {
      "id": "uuid",
      "name": "Laptops",
      "slug": "laptops"
    },
    "createdAt": "2024-01-01T00:00:00.000Z",
    "updatedAt": "2024-01-01T00:00:00.000Z"
  }
}
```

Postman Testing Steps:

1. Create a new GET request
2. Enter URL: `http://localhost:3000/api/v1/products/{product_id}`
3. Replace `{product_id}` with a valid product UUID
4. Click **Send**
5. Verify product details are returned
6. Expected status: `200 OK`

Error Scenario:

- Use invalid UUID → Expected: `404 Not Found`

---

4. Create Product (Vendor)

Endpoint: `POST /api/v1/products`

Description: Create a new product. Only authenticated vendors can create products. The vendor ID is automatically set from the authenticated user. SKU must be unique across all products. Slug is auto-generated from the product name.

Authentication: Required (Vendor role)

Request Body:

```
{
  "sku": "PROD-001",
  "name": "Laptop Pro",
  "description": "High-performance laptop for professionals",
  "basePrice": 999.99,
  "salePrice": 899.99,
  "unit": "each",
  "minOrderQty": 1,
  "maxOrderQty": 10,
  "stockQty": 50,
  "categoryId": "uuid",
  "tags": ["laptop", "computer", "pro"],
  "attributes": {
    "brand": "TechBrand",
```

```
    "warranty": "2 years"
  },
  "status": "DRAFT",
  "isFeatured": false
}
```

Validation Rules:

- `sku`: Required, unique, string
- `name`: Required, min 2 characters
- `basePrice`: Required, number > 0
- `salePrice`: Optional, must be < basePrice
- `unit`: Required, min 1 character
- `stockQty`: Required, integer >= 0
- `minOrderQty`: Optional, default 1, >= 1
- `maxOrderQty`: Optional, >= minOrderQty
- `categoryId`: Optional, must exist
- `status`: Optional, default "DRAFT", enum: ACTIVE, INACTIVE, DRAFT, DISCONTINUED

Response:

```
{
  "success": true,
  "data": {
    "id": "uuid",
    "sku": "PROD-001",
    "name": "Laptop Pro",
    "slug": "laptop-pro",
    "description": "High-performance laptop for professionals",
    "basePrice": 999.99,
    "salePrice": 899.99,
    "unit": "each",
    "minOrderQty": 1,
    "maxOrderQty": 10,
    "stockQty": 50,
    "stockStatus": "IN_STOCK",
    "images": [],
    "attributes": {
      "brand": "TechBrand",
      "warranty": "2 years"
    },
    "tags": ["laptop", "computer", "pro"],
    "status": "DRAFT",
    "isFeatured": false,
    "vendor": {
      "id": "uuid",
      "name": "TechStore",
      "slug": "techstore"
```

```
    },
    "category": {
      "id": "uuid",
      "name": "Laptops",
      "slug": "laptops"
    },
    "createdAt": "2024-01-01T00:00:00.000Z",
    "updatedAt": "2024-01-01T00:00:00.000Z"
  }
}
```

Postman Testing Steps:

1. **First, authenticate as vendor:**
   - Create POST request to `http://localhost:3000/api/v1/auth/login`
   - Body: `{ "email": "vendor@example.com", "password": "password123" }`
   - Copy the `token` from response
2. **Create product:**
   - Create new POST request
   - Enter URL: `http://localhost:3000/api/v1/products`
   - Go to **Headers** tab and add:
     - Key: `Authorization`
     - Value: `Bearer {your_token}`
   - Go to **Body** tab, select **raw** and **JSON**
   - Paste the request body JSON above
   - Click **Send**
   - Verify product is created with auto-generated slug
   - Expected status: `201 Created`

Error Scenarios:

- Missing required fields → `400 Bad Request` with validation errors
- Duplicate SKU → `400 Bad Request` with "SKU must be unique"
- Invalid categoryId → `400 Bad Request` with "Category not found"
- Not authenticated → `401 Unauthorized`
- Not a vendor → `403 Forbidden`

---

5. Update Product (Vendor)

Endpoint: `PUT /api/v1/products/:id`

Description: Update an existing product. Vendors can only update their own products. If status changes to ACTIVE, `publishedAt` is automatically set to current date. Slug is regenerated if name changes.

Authentication: Required (Vendor role)

Path Parameters:

- `id` (string, required): Product UUID

Request Body:

```
{
  "name": "Laptop Pro Updated",
  "description": "Updated description",
  "basePrice": 1099.99,
  "salePrice": 999.99,
  "unit": "each",
  "minOrderQty": 1,
  "maxOrderQty": 15,
  "stockQty": 45,
  "categoryId": "uuid",
  "status": "ACTIVE",
  "isFeatured": true
}
```

Response:

```
{
  "success": true,
  "data": {
    "id": "uuid",
    "sku": "PROD-001",
    "name": "Laptop Pro Updated",
    "slug": "laptop-pro-updated",
    "description": "Updated description",
    "basePrice": 1099.99,
    "salePrice": 999.99,
    "unit": "each",
    "minOrderQty": 1,
    "maxOrderQty": 15,
    "stockQty": 45,
    "stockStatus": "IN_STOCK",
    "images": [],
    "attributes": {},
    "tags": [],
    "status": "ACTIVE",
    "isFeatured": true,
    "vendor": {
      "id": "uuid",
      "name": "TechStore",
      "slug": "techstore"
    },
    "category": {
```

```
      "id": "uuid",
      "name": "Laptops",
      "slug": "laptops"
    },
    "createdAt": "2024-01-01T00:00:00.000Z",
    "updatedAt": "2024-01-02T00:00:00.000Z",
    "publishedAt": "2024-01-02T00:00:00.000Z"
  }
}
```

Postman Testing Steps:

1. Use the vendor token from previous step
2. Create new PUT request
3. Enter URL: `http://localhost:3000/api/v1/products/{product_id}`
4. Add Authorization header with Bearer token
5. In **Body** tab (raw JSON), paste the request body
6. Click **Send**
7. Verify product is updated
8. Expected status: `200 OK`

Error Scenarios:

- Product not found → `404 Not Found`
- Not product owner → `403 Forbidden` with "Unauthorized: You can only update your own products"
- Slug conflict → `400 Bad Request` with "Product slug conflict"

---

6. Delete Product (Vendor)

Endpoint: `DELETE /api/v1/products/:id`

Description: Soft delete a product by setting `deletedAt` timestamp. Deleted products are excluded from all search queries by default. Vendors can only delete their own products.

Authentication: Required (Vendor role)

Path Parameters:

- `id` (string, required): Product UUID

Response:

```
{
  "success": true,
  "message": "Product deleted successfully"
}
```

Postman Testing Steps:

1. Use the vendor token
2. Create new DELETE request
3. Enter URL: `http://localhost:3000/api/v1/products/{product_id}`
4. Add Authorization header with Bearer token
5. Click **Send**
6. Expected status: `200 OK`

Verification:

- Try to get the product via GET endpoint → Should return `404 Not Found`
- Try to search products → Deleted product should not appear in results

Error Scenarios:

- Product not found → `404 Not Found`
- Not product owner → `403 Forbidden`

---

7. Restore Product (Vendor)

Endpoint: `POST /api/v1/products/:id/restore`

Description: Restore a soft-deleted product by setting `deletedAt` to null. Vendors can only restore their own products.

Authentication: Required (Vendor role)

Path Parameters:

- `id` (string, required): Product UUID

Response:


```
{
  "success": true,
  "data": {
    "id": "uuid",
    "sku": "PROD-001",
    "name": "Laptop Pro Updated",
    "slug": "laptop-pro-updated",
    "description": "Updated description",
    "basePrice": 1099.99,
    "salePrice": 999.99,
    "unit": "each",
    "minOrderQty": 1,
    "maxOrderQty": 15,
    "stockQty": 45,
    "stockStatus": "IN_STOCK",
```

```
    "images": [],
    "attributes": {},
    "tags": [],
    "status": "ACTIVE",
    "isFeatured": true,
    "vendor": {
      "id": "uuid",
      "name": "TechStore",
      "slug": "techstore"
    },
    "category": {
      "id": "uuid",
      "name": "Laptops",
      "slug": "laptops"
    },
    "createdAt": "2024-01-01T00:00:00.000Z",
    "updatedAt": "2024-01-02T00:00:00.000Z"
  }
}
```

Postman Testing Steps:

1. Use the vendor token
2. Create new POST request
3. Enter URL: `http://localhost:3000/api/v1/products/{product_id}/restore`
4. Add Authorization header with Bearer token
5. Click **Send**
6. Expected status: `200 OK`

Verification:

- Try to get the product via GET endpoint → Should return product data

Error Scenarios:

- Product not found → `404 Not Found`
- Not product owner → `403 Forbidden`

---

8. Upload Product Images (Vendor)

Endpoint: `POST /api/v1/products/:id/images`

Description: Upload multiple images for a product. Images are stored on Cloudinary and URLs are appended to the product's images array. Maximum 10 images per request, 5MB each. Supported formats: JPG, PNG, WebP.

Authentication: Required (Vendor role)

Path Parameters:

- `id` (string, required): Product UUID

Request Body:

- **form-data** with key `images` (type: File)
- Can upload multiple files (up to 10)

Response:


```
{
  "success": true,
  "data": {
    "id": "uuid",
    "sku": "PROD-001",
    "name": "Laptop Pro Updated",
    "slug": "laptop-pro-updated",
    "description": "Updated description",
    "basePrice": 1099.99,
    "salePrice": 999.99,
    "unit": "each",
    "minOrderQty": 1,
    "maxOrderQty": 15,
    "stockQty": 45,
    "stockStatus": "IN_STOCK",
    "images": [
      "https://res.cloudinary.com/your-cloud/image1.jpg",
      "https://res.cloudinary.com/your-cloud/image2.jpg"
    ],
    "attributes": {},
    "tags": [],
    "status": "ACTIVE",
    "isFeatured": true,
    "vendor": {
      "id": "uuid",
      "name": "TechStore",
      "slug": "techstore"
    },
    "category": {
      "id": "uuid",
      "name": "Laptops",
      "slug": "laptops"
    },
    "createdAt": "2024-01-01T00:00:00.000Z",
    "updatedAt": "2024-01-02T00:00:00.000Z"
  }
}
```


Postman Testing Steps:

1. Use the vendor token
2. Create new POST request
3. Enter URL: `http://localhost:3000/api/v1/products/{product_id}/images`
4. Add Authorization header with Bearer token

5. Go to **Body** tab
6. Select **form-data**
7. Add key: `images` (type: File)
8. Select one or more image files from your computer
9. Click **Send**
10. Verify images are added to product
11. Expected status: `200 OK`

Error Scenarios:

- Product not found → `404 Not Found`
- Not product owner → `403 Forbidden`
- File too large (>5MB) → `400 Bad Request`
- Invalid file type → `400 Bad Request`

Admin Endpoints

Base URL: `http://localhost:3000/api/admin/v1/products`

Note: All admin endpoints require authentication with ADMIN or SUPER_ADMIN role.

1. Create Product (Admin)

Endpoint: `POST /api/admin/v1/products`

Description: Create a new product as an admin. Admins can specify the vendor ID to create products on behalf of vendors. All other functionality is the same as vendor product creation.

Authentication: Required (ADMIN or SUPER_ADMIN role)

Request Body:

```
{
  "sku": "ADMIN-PROD-001",
  "name": "Admin Laptop",
  "description": "Product created by admin",
  "basePrice": 1299.99,
  "salePrice": 1199.99,
  "unit": "each",
  "minOrderQty": 1,
  "maxOrderQty": 20,
  "stockQty": 100,
  "vendorId": "vendor-uuid",
  "categoryId": "category-uuid",
  "tags": ["admin", "laptop"],
  "status": "ACTIVE",
```

```
  "isFeatured": true
}
```

Additional Field (Admin only):

- `vendorId` (string, required): The vendor ID to assign the product to

Response: Same as vendor create product response

Postman Testing Steps:

1. **First, authenticate as admin:**
    - Create POST request to `http://localhost:3000/api/v1/auth/login`
    - Body: `{ "email": "admin@example.com", "password": "admin123" }`
    - Copy the `token` from response
2. **Create product:**
    - Create new POST request
    - Enter URL: `http://localhost:3000/api/admin/v1/products`
    - Add Authorization header with Bearer token
    - In **Body** tab (raw JSON), paste the request body with valid vendorId
    - Click **Send**
    - Expected status: `201 Created`

Error Scenarios:

- Invalid vendorId → `400 Bad Request` with "Vendor not found"
- Duplicate SKU → `400 Bad Request`
- Not admin → `403 Forbidden`

---

2. Update Product (Admin)

Endpoint: `PUT /api/admin/v1/products/:id`

Description: Update any product in the system. Admins can update products from any vendor. All other functionality is the same as vendor product update.

Authentication: Required (ADMIN or SUPER_ADMIN role)

Path Parameters:

- `id` (string, required): Product UUID

Request Body: Same as vendor update (without vendor restriction)

Response: Same as vendor update product response

Postman Testing Steps:

1. Use the admin token
2. Create new PUT request
3. Enter URL: `http://localhost:3000/api/admin/v1/products/{product_id}`
4. Add Authorization header with Bearer token
5. In **Body** tab (raw JSON), paste the update data
6. Click **Send**
7. Expected status: `200 OK`

Error Scenarios:

- Product not found → `404 Not Found`
- Not admin → `403 Forbidden`

---

3. Get Product by ID (Admin)

Endpoint: `GET /api/admin/v1/products/:id`

Description: Retrieve a single product by ID. Admins can view any product including soft-deleted ones (if they exist in database). Results are cached for 5 minutes.

Authentication: Required (ADMIN or SUPER_ADMIN role)

Path Parameters:

- `id` (string, required): Product UUID

Response: Same as public get product response

Postman Testing Steps:

1. Use the admin token
2. Create new GET request
3. Enter URL: `http://localhost:3000/api/admin/v1/products/{product_id}`
4. Add Authorization header with Bearer token
5. Click **Send**
6. Expected status: `200 OK`

---

4. Delete Product (Admin)

Endpoint: `DELETE /api/admin/v1/products/:id`

Description: Soft delete any product in the system. Admins can delete products from any vendor.

Authentication: Required (ADMIN or SUPER_ADMIN role)

Path Parameters:

- `id` (string, required): Product UUID

Response:

```
{
  "success": true,
  "message": "Product deleted successfully"
}
```

Postman Testing Steps:

1. Use the admin token
2. Create new DELETE request
3. Enter URL: `http://localhost:3000/api/admin/v1/products/{product_id}`
4. Add Authorization header with Bearer token
5. Click **Send**
6. Expected status: `200 OK`

---

5. Restore Product (Admin)

Endpoint: `PATCH /api/admin/v1/products/:id/restore`

Description: Restore any soft-deleted product. Admins can restore products from any vendor. Uses PATCH method as per specification.

Authentication: Required (ADMIN or SUPER_ADMIN role)

Path Parameters:

- `id` (string, required): Product UUID

Response: Same as vendor restore product response

Postman Testing Steps:

1. Use the admin token
2. Create new PATCH request
3. Enter URL: `http://localhost:3000/api/admin/v1/products/{product_id}/restore`
4. Add Authorization header with Bearer token
5. Click **Send**
6. Expected status: `200 OK`

---

6. Search Products (Admin)

Endpoint: `GET /api/admin/v1/products`

Description: Search and filter products with admin-level access. Includes all the same filtering options as public search but with admin authentication. Results are cached for 5 minutes.

Authentication: Required (ADMIN or SUPER_ADMIN role)

Query Parameters: Same as public search products

Response: Same as public search products response

Postman Testing Steps:

1. Use the admin token
2. Create new GET request
3. Enter URL: `http://localhost:3000/api/admin/v1/products`
4. Add query parameters as needed (e.g., status=DRAFT, vendorId=uuid)
5. Add Authorization header with Bearer token
6. Click **Send**
7. Expected status: `200 OK`

Use Cases:

- View all draft products
- View products by specific vendor
- View soft-deleted products (if query allows)
- Admin-level filtering and sorting

---

7. Get Vendor Products (Admin)

Endpoint: `GET /api/admin/v1/products/vendor/:vendorId`

Description: Retrieve all products for a specific vendor. Useful for admin to review a vendor's product catalog.

Authentication: Required (ADMIN or SUPER_ADMIN role)

Path Parameters:

- `vendorId` (string, required): Vendor UUID

Query Parameters: Same as search products (except vendorId is from path)

Response: Same as search products response (filtered by vendor)

Postman Testing Steps:

1. Use the admin token
2. Create new GET request
3. Enter URL: `http://localhost:3000/api/admin/v1/products/vendor/{vendor_id}`
4. Add optional query parameters (status, page, limit, etc.)
5. Add Authorization header with Bearer token
6. Click **Send**
7. Expected status: `200 OK`

Authentication Setup

Before testing protected endpoints, you need to authenticate and obtain a JWT token.

For Vendor Role:

1. **Register a new vendor (if needed):**
    - POST `http://localhost:3000/api/v1/auth/vendor/register`
    - Body:

```
{
"name": "Test Vendor",
"email": "vendor@example.com",
"password": "password123",
"phone": "1234567890",
"businessName": "Test Business",
"businessType": "RETAILER"
}
```

2. **Login as vendor:**
    - POST `http://localhost:3000/api/v1/auth/login`
    - Body:

```
{
"email": "vendor@example.com",
"password": "password123"
}
```

    - Copy the `token` from response

For Admin Role:

3. **Login as admin:**
    - POST `http://localhost:3000/api/v1/auth/login`
    - Body:

```
{
"email": "admin@example.com",
"password": "admin123"
}
```

- Copy the `token` from response

Using the Token:

For all protected endpoints:

4. Go to **Headers** tab in Postman
5. Add a new header:
   - Key: `Authorization`
   - Value: `Bearer {your_token_here}`
6. Replace `{your_token_here}` with the actual token from login response

Common Error Responses

400 Bad Request

```
{
  "success": false,
  "message": "Validation failed",
  "errors": [
    {
      "path": ["name"],
      "message": "Required"
    }
  ]
}
```

401 Unauthorized

```
{
  "success": false,
  "message": "Authentication required"
}
```

403 Forbidden

```
{
  "success": false,
  "message": "Insufficient permissions"
}
```

404 Not Found

```
{
  "success": false,
  "message": "Product not found"
}
```

500 Internal Server Error

```
{
  "success": false,
  "message": "An error occurred"
}
```

Testing Checklist

Public Endpoints:

- [ ] Get category tree
- [ ] Search products with various filters
- [ ] Get product by ID
- [ ] Search with category + subcategories
- [ ] Search with multiple vendor IDs

Vendor Endpoints:

- [ ] Create product
- [ ] Update own product
- [ ] Delete own product
- [ ] Restore own product
- [ ] Upload product images
- [ ] Attempt to update another vendor's product (should fail)

Admin Endpoints:

- [ ] Create product for vendor
- [ ] Update any product
- [ ] Delete any product
- [ ] Restore any product
- [ ] Search all products
- [ ] Get vendor products
- [ ] View draft products

Notes

- **Caching:** Category tree is cached for 1 hour, product searches for 5 minutes

- **Soft Delete:** Deleted products are not returned in search queries but can be restored
- **Stock Status:** Auto-calculated based on stockQty:
    - stockQty > 10: IN_STOCK
    - 1 <= stockQty <= 10: LOW_STOCK
    - stockQty == 0: OUT_OF_STOCK
    - Manual BACKORDER status overrides auto-calculation
- **Slug Generation:** Auto-generated from product name (lowercase, hyphens instead of spaces)
- **Image Storage:** Images are stored on Cloudinary (not local filesystem)
- **Pagination:** Default page size is 20, maximum is 100

Comprehensive Endpoint Reference

Quick Reference Table

Method	Endpoint	Auth Required	Role	Description
Public Endpoints				
GET	`/api/v1/categories`	No	Public	Get category tree with product counts
GET	`/api/v1/products`	No	Public	Search and filter products
GET	`/api/v1/products/:id`	No	Public	Get product by ID
Vendor Endpoints				
POST	`/api/v1/products`	Yes	VENDOR	Create new product
PUT	`/api/v1/products/:id`	Yes	VENDOR	Update own product
DELETE	`/api/v1/products/:id`	Yes	VENDOR	Delete own product

				(soft delete)
POST	`/api/v1/products/:id/restore`	Yes	VENDOR	Restore own product
POST	`/api/v1/products/:id/images`	Yes	VENDOR	Upload product images
Admin Endpoints				
POST	`/api/admin/v1/products`	Yes	ADMIN, SUPER_ADMIN	Create product for any vendor
PUT	`/api/admin/v1/products/:id`	Yes	ADMIN, SUPER_ADMIN	Update any product
GET	`/api/admin/v1/products/:id`	Yes	ADMIN, SUPER_ADMIN	Get any product by ID
DELETE	`/api/admin/v1/products/:id`	Yes	ADMIN, SUPER_ADMIN	Delete any product
PATCH	`/api/admin/v1/products/:id/restore`	Yes	ADMIN, SUPER_ADMIN	Restore any product
GET	`/api/admin/v1/products`	Yes	ADMIN, SUPER_ADMIN	Search all products
GET	`/api/admin/v1/products/vendor/:vendorId`	Yes	ADMIN, SUPER_ADMIN	Get vendor products

Endpoint Details by Category

Category Management

- **Get Category Tree**: `GET /api/v1/categories`
    - Purpose: Retrieve hierarchical category structure
    - Cache: 1 hour TTL
    - Response: Nested tree with product counts

Product Search & Discovery

- **Search Products**: `GET /api/v1/products`
    - Purpose: Advanced product search with filters

    - Cache: 5 minutes TTL
    - Filters: category, vendor, price, stock, featured, status
    - Sorting: createdAt, basePrice, name
    - Pagination: page, limit (max 100)
- **Get Product by ID**: `GET /api/v1/products/:id`
    - Purpose: Retrieve single product details
    - Cache: 5 minutes TTL
    - Includes: Vendor and category information

Vendor Product Management

- **Create Product**: `POST /api/v1/products`
    - Purpose: Create new product for vendor
    - Auth: Vendor JWT required
    - Validation: SKU uniqueness, category existence
    - Auto-generated: Slug, stock status
    - Restrictions: Can only create for own vendor
- **Update Product**: `PUT /api/v1/products/:id`
    - Purpose: Update existing product
    - Auth: Vendor JWT required
    - Restrictions: Can only update own products
    - Auto-updated: Slug (if name changed), publishedAt (if status=ACTIVE)
- **Delete Product**: `DELETE /api/v1/products/:id`
    - Purpose: Soft delete product
    - Auth: Vendor JWT required
    - Restrictions: Can only delete own products
    - Method: Sets deletedAt timestamp
- **Restore Product**: `POST /api/v1/products/:id/restore`
    - Purpose: Restore soft-deleted product
    - Auth: Vendor JWT required
    - Restrictions: Can only restore own products
    - Method: Sets deletedAt to null

- **Upload Images**: `POST /api/v1/products/:id/images`
    - Purpose: Upload product images
    - Auth: Vendor JWT required
    - Storage: Cloudinary
    - Limits: 10 images, 5MB each
    - Formats: JPG, PNG, WebP

Admin Product Management

- **Admin Create Product**: `POST /api/admin/v1/products`
    - Purpose: Create product for any vendor
    - Auth: Admin/Super Admin JWT required
    - Additional field: vendorId (required)
    - Same validation as vendor creation
- **Admin Update Product**: `PUT /api/admin/v1/products/:id`
    - Purpose: Update any product
    - Auth: Admin/Super Admin JWT required
    - No vendor restrictions
    - Same validation as vendor update
- **Admin Get Product**: `GET /api/admin/v1/products/:id`
    - Purpose: Get any product by ID
    - Auth: Admin/Super Admin JWT required
    - Can view soft-deleted products
- **Admin Delete Product**: `DELETE /api/admin/v1/products/:id`
    - Purpose: Delete any product
    - Auth: Admin/Super Admin JWT required
    - No vendor restrictions
- **Admin Restore Product**: `PATCH /api/admin/v1/products/:id/restore`
    - Purpose: Restore any deleted product
    - Auth: Admin/Super Admin JWT required
    - No vendor restrictions
- **Admin Search Products**: `GET /api/admin/v1/products`

    - Purpose: Search all products with admin access
    - Auth: Admin/Super Admin JWT required
    - Same filters as public search
    - Can view draft/inactive products
- **Admin Get Vendor Products**: `GET /api/admin/v1/products/vendor/:vendorId`
    - Purpose: Get all products for specific vendor
    - Auth: Admin/Super Admin JWT required
    - Purpose: Vendor catalog review

Authentication & Authorization Flow

```
1. User Login
   POST /api/v1/auth/login
   → Returns JWT token

2. Token Usage
   Authorization: Bearer {token}

3. Middleware Chain
   Request → authenticate() → authorize(role) → Controller → Service → Database

4. Role-Based Access
   - Public: No auth required
   - Vendor: authenticate + authorize("VENDOR")
   - Admin: authenticate + authorize("ADMIN", "SUPER_ADMIN")
```

Error Handling Reference

Status Code	Error Type	Description
200	Success	Request completed successfully
201	Created	Resource created successfully
400	Bad Request	Validation failed, invalid data
401	Unauthorized	Missing or invalid authentication
403	Forbidden	Insufficient permissions
404	Not Found	Resource not found
500	Internal Server Error	Server error occurred

Cache Strategy

Cache Key	TTL	Invalidation
`categories:tree`	1 hour	On category CRUD operations

`categories:descendants:{id}`	30 min	On category CRUD operations
`product:{id}`	5 min	On product CRUD operations
`products:search:*`	5 min	On product CRUD operations
`admin:products:search:*`	5 min	On product CRUD operations

Database Schema Relationships

User → Vendor → Product → Category

- User (1) → (1) Vendor
- Vendor (1) → (N) Product
- Category (1) → (N) Product
- Category (1) → (N) Category (self-referencing for hierarchy)

Key Foreign Keys:

- `products.vendor_id` → `vendors.id`
- `products.category_id` → `categories.id`
- `vendors.user_id` → `users.id`
- `categories.parent_id` → `categories.id`

Future Enhancements

Potential features for future implementation:

- Category management endpoints (CRUD)
- Bulk product operations for vendors